DB 덤프 Release 사용법

덤프 파일을 왜 브랜치가 아니라 GitHub Release에 올렸는지, 그리고 각자 자기 브랜치에서 그 덤프로 DB를 똑같이 맞추는 방법을 정리했습니다. Git이나 DB를 처음 다루는 사람도 순서대로 따라 하면 됩니다.

날짜	2026-10-06
최신 덤프	`cheongyak_20261006_2.dump` (66MB, main 커밋 `8463dbb` 기준)
받는 곳	github.com/JiYoon241111/cheongyak-platform/releases/tag/db-dump-20261006-2
관련 문서	`docs/261002_7_청약플랫폼_덤프파일_가이드.pdf` · `docs/261006_8_청약플랫폼_수집스케줄러_백업_변경사항.pdf`

목차

1 **한눈에 보기**
덤프·Release가 무엇인지 세 줄 요약, 바로 따라 할 명령

2 **덤프 파일과 Release란?** 개념
브랜치(코드)와 Release(첨부 파일)의 차이를 비유로

3 **왜 브랜치에 커밋하지 않고 Release에 올렸나** 이유
파일 크기, Git 기록, .gitignore 규칙

4 **내 브랜치에서 덤프 사용하기** 따라 하기
받기 → 복원 → 확인, 단계별 명령

5 **브랜치별 주의할 점**
main · jiyoon · 그 밖의 팀원 브랜치

6 **자주 묻는 질문**
Release가 안 보여요, 내 데이터는 어떻게 되나요 등

1. 한눈에 보기

덤프

DB 전체(테이블 구조 + 데이터)를 파일 하나로 저장한 것. 복원하면 내 DB가 덤프를 만든 시점과 똑같아짐

Release

GitHub 저장소의 "자료 게시판". 특정 시점에 글을 올리고 큰 파일을 첨부해 두는 곳

브랜치 무관

Release는 저장소 전체가 같이 쓰므로, 어느 브랜치에서 작업하든 같은 덤프를 받아 쓰면 됨

> **요약:** 덤프 파일은 코드가 아니라 **DB 내용**이라서 브랜치에 넣지 않고 Release에 첨부했습니다. 각자 Release에서 파일을 받아 **내 컴퓨터의 DB 컨테이너에 복원**하면 됩니다. 브랜치를 바꾸거나 합칠 필요는 없습니다.

바로 따라 하기 (PowerShell, 프로젝트 폴더에서)

Release 페이지에서 `cheongyak_20261006_2.dump`를 받아 **cheongyak-platform 폴더**에 넣은 뒤:

```
docker compose up -d db
docker compose stop backend
docker compose cp .\cheongyak_20261006_2.dump db:/tmp/cheongyak.dump
docker compose exec -T db psql -U cheongyak -d postgres -c "DROP DATABASE IF EXISTS cheongyak WITH (FORCE)" -c
"CREATE DATABASE cheongyak"
docker compose exec -T db pg_restore -U cheongyak -d cheongyak --no-owner /tmp/cheongyak.dump
docker compose exec -T db rm /tmp/cheongyak.dump
docker compose up -d --build backend
```

각 줄이 무슨 뜻인지는 4장에 자세히 있습니다. 복원은 약 40초 걸리고, 아무 메시지 없이 끝나면 성공입니다.

이번 덤프에 들어 있는 것

항목	내용
공고 (listings)	청약홈 25,899행 · LH 100,212행 · SH 22행
계약금	접수 중·예정 공고 369행에 계약금 (팀원 방식: 공고문에 적힌 숫자만, 10%로 채우지 않음)
임대보증금	임대보증금만 있는 공고는 계약금 칸 대신 `rent_deposit`(보증금 칸)에
그 밖의 테이블	경쟁률·당첨가점, 대출 상품, 청약 통계, 공고문 원문, LH 주택목록, 수집 기록 등 15개 테이블 + 가공 뷰 3개
Alembic	`alembic_version = 002_user_addresses` (팀원의 사용자 주소 컬럼까지 반영된 상태)

2026-10-06 실제 확인: 빈 DB에 복원 → 오류 없이 36초, 위 행 수와 계약금 369행이 그대로 나옴.

2. 덤프 파일과 Release란?

2-1. 덤프 파일 = DB를 통째로 찍은 사진

우리 DB(PostgreSQL)는 Docker의 `db` 컨테이너 안에 있고, **각자 컴퓨터마다 따로** 있습니다. 코드는 Git으로 공유되지만, DB 안의 데이터는 Git으로 공유되지 않습니다.

그래서 데이터를 처음부터 모으려면 전체 수집을 직접 돌려야 하는데, LH API는 하루 호출 한도가 있어서 2010년부터의 공고를 다 받는 데 **며칠**이 걸립니다. 덤프 파일은 이미 다 모은 DB를 `pg_dump`로 파일 하나에 저장한 것이라, 받아서 복원하면 **1분 안에** 같은 데이터를 가질 수 있습니다.

```
[지윤 컴퓨터의 DB]  ──pg_dump──▶  cheongyak_20261006_2.dump  ──Release에 첨부──▶  GitHub
                                                                                    │
[팀원 컴퓨터의 DB]  ◀──pg_restore──  cheongyak_20261006_2.dump  ◀──다운로드──────────┘
```

2-2. Release = 저장소의 자료 게시판

	브랜치 (main, jiyoon ...)	Release
비유	함께 고쳐 나가는 공동 문서. 고칠 때마다 기록(커밋)이 쌓임	게시판에 올린 글 + 첨부 파일. 올린 시점 그대로 보관
담는 것	코드, 설정, 문서(PDF) 같은 작은 파일	덤프, 설치 파일처럼 큰 파일 (파일당 2GB까지)
받는 방법	`git pull`, `git clone` 하면 자동으로 받아짐	필요한 사람만 웹 페이지에서 직접 클릭해 다운로드
범위	브랜치마다 내용이 다름	**저장소 전체 공용**. 어느 브랜치에서 작업하든 같은 Release를 씀
찾는 곳	저장소 첫 화면의 브랜치 선택 메뉴	저장소 첫 화면 오른쪽의 **Releases**, 또는 주소 뒤에 `/releases`

2-3. Release 하나의 구성

구성	이번 덤프의 값	뜻
태그 (tag)	`db-dump-20261006-2`	어느 커밋 시점의 자료인지 표시하는 이름표
연결된 커밋	main `8463dbb`	이 덤프와 짝이 맞는 코드 시점 (jiyoon 브랜치 내용도 이 커밋에 합쳐져 있음)
제목·설명	DB 덤프 2026-10-06 (계약금 반영)	무엇이 들어 있는지, 복원 명령
첨부 파일 (Assets)	`cheongyak_20261006_2.dump`	실제로 받을 파일. Assets 목록에서 클릭

> **Assets 목록에 "Source code (zip)"도 같이 보입니다.** GitHub가 자동으로 붙이는 그 시점의 코드 압축 파일이라 덤프와 상관없습니다. `.dump`로 끝나는 파일만 받으면 됩니다.

3. 왜 브랜치에 커밋하지 않고 Release에 올렸나

이유	브랜치에 커밋하면	Release에 올리면
① 파일 크기	덤프는 66MB. GitHub는 50MB가 넘는 파일에 경고하고, 100MB가 넘으면 아예 거부함. 데이터가 늘면 곧 막힘	파일당 2GB까지 올릴 수 있음
② 지워지지 않는 기록	Git은 모든 버전을 영원히 보관함. 나중에 파일을 지워도 기록에 남아서, 덤프를 새로 올릴 때마다 저장소가 66MB씩 계속 무거워짐	저장소 기록과 별개라 코드 저장소 크기에 영향 없음. 오래된 덤프는 지우면 정말 사라짐
③ 모두가 강제로 받음	`git clone` · `git pull` 할 때 덤프가 필요 없는 팀원(프론트 작업 등)도 전부 받아야 함	필요한 사람만, 필요할 때 받음
④ 팀 규칙	팀 저장소 `.gitignore` 에 `*.dump` 가 있어서 Git이 덤프 파일을 아예 무시하도록 정해 둠 (커밋 `1f1e8a9`)	규칙과 충돌 없음
⑤ 코드와 데이터는 따로 바뀜	데이터는 매일 오전 6시 수집으로 바뀌는데, 그때마다 커밋하면 코드 변경 기록 사이에 데이터 커밋이 섞임	필요한 시점에만 새 Release를 추가

> **정리:** 브랜치는 **코드**를 함께 고치는 곳이고, 덤프는 **데이터**를 나눠 주는 파일입니다. 그래서 코드는 브랜치로, 덤프는 Release로 나눴습니다. 팀 문서(PDF)처럼 작은 파일은 지금처럼 `docs/` 폴더에 커밋합니다.

덤프 파일을 받은 뒤 Git에 올라가지 않나요?

받은 `.dump` 파일을 프로젝트 폴더에 그대로 두어도 괜찮습니다. `.gitignore` 의 `*.dump` 규칙 때문에 `git status` 에 나오지 않고, `git add .` 를 해도 커밋되지 않습니다.

지금까지 올린 덤프 (팀 저장소 · 개인 저장소 공통)

파일	만든 날	특징
`cheongyak_20261006_2.dump`	2026-10-06 오후	**최신. 이것을 쓰세요.** main에 합친 뒤, 계약금(팀원 방식) 채움
`cheongyak_20261006.dump`	2026-10-06 오전	계약금 칸 반영 전 (개인 저장소에만)
`cheongyak_20261002.dump`	2026-10-02	처음 만든 덤프 (개인 저장소에만)

4. 내 브랜치에서 덤프 사용하기

> **기억할 것:** 덤프 복원은 **DB만** 바꿉니다. 내 브랜치의 코드, 커밋, 작업 중인 파일은 전혀 바뀌지 않습니다. 그래서 브랜치를 바꾸지 않고 지금 작업하던 브랜치에서 그대로 하면 됩니다.

단계 1. 덤프 파일 받기

1. GitHub에 **팀 저장소 멤버 계정으로 로그인**합니다. (저장소가 비공개라 로그인해야 보입니다.)
2. `github.com/JiYoon241111/cheongyak-platform` → 오른쪽 **Releases** → **DB 덤프 2026-10-06 (계약금 반영)**을 엽니다.
3. 아래쪽 **Assets**에서 `cheongyak_20261006_2.dump`를 클릭해 받습니다.
4. 받은 파일을 **cheongyak-platform 폴더**(`docker-compose.yml`이 있는 폴더)로 옮깁니다.

단계 2. 복원하기 (PowerShell, cheongyak-platform 폴더에서)

```
# ① DB 컨테이너만 켜고, DB를 쓰고 있는 backend는 잠시 멈춤
docker compose up -d db
docker compose stop backend

# ② 덤프 파일을 DB 컨테이너 안으로 복사
docker compose cp .\cheongyak_20261006_2.dump db:/tmp/cheongyak.dump

# ③ 기존 DB를 지우고 빈 DB 만들기  ← 내 DB의 기존 데이터가 지워짐 (6장 참고)
docker compose exec -T db psql -U cheongyak -d postgres -c "DROP DATABASE IF EXISTS cheongyak WITH (FORCE)" -c
"CREATE DATABASE cheongyak"

# ④ 복원 (약 40초. 아무 메시지 없이 끝나면 성공)
docker compose exec -T db pg_restore -U cheongyak -d cheongyak --no-owner /tmp/cheongyak.dump

# ⑤ 임시 파일 지우고 backend 다시 켜기 (--build: 내 브랜치 코드의 라이브러리 설치)
docker compose exec -T db rm /tmp/cheongyak.dump
docker compose up -d --build backend
```

> **Git Bash에서 할 때:** ②번처럼 `/tmp/…` 경로가 들어간 명령 앞에 `MSYS_NO_PATHCONV=1`을 붙이세요. Git Bash가 `/tmp`를 Windows 경로로 바꿔 버려서 "파일을 찾을 수 없음" 오류가 납니다. PowerShell에서는 필요 없습니다.

단계 3. 잘 들어갔는지 확인

```
docker compose exec -T db psql -U cheongyak -d cheongyak -c "SELECT source, count(*) FROM listings GROUP BY
source"
```

source	나와야 하는 행 수
applyhome	25,899
lh	100,212
sh	22

브라우저에서 `http://localhost:8000/docs`를 열어 `GET /listings/`를 실행해 봐도 됩니다. 복원 뒤에는 매일 오전 6시 수집이 내 컴퓨터에서 이어서 돌며 최신 공고를 더합니다.

5. 브랜치별 주의할 점

덤프에는 **main 커밋 `8463dbb` 기준의 테이블 구조**가 들어 있습니다. 내 브랜치 코드가 이 구조를 알고 있는지에 따라 아래처럼 다릅니다.

내 브랜치	그대로 사용	설명
`main`	가능	덤프와 짝이 맞는 코드. 계약금, 사용자 주소 컬럼, 챗봇·통근 API 모두 그대로 동작
`jiyoon(cheongyakP)`	가능	main에 합쳐진 DB 수집 코드가 이 브랜치에서 온 것이라 테이블이 맞음. 덤프의 계약금 칸(`deposit_*`)은 이 브랜치 코드가 쓰지 않을 뿐 문제 없음
main 이전에 갈라진 팀원 브랜치 (`feat/frontend` , `feat/llm` 등)	main 받은 뒤	예전 DB 구조를 기준으로 짠 코드라 일부 컬럼 이름이 다를 수 있음. 아래처럼 **main을 내 브랜치에 먼저 합친 뒤** 복원하는 것을 권장

내 브랜치에 main 받아오기 (복원 전에)

```
git status                # 작업 중인 파일이 있으면 먼저 커밋
git fetch origin
git merge origin/main     # 내 브랜치에 최신 main을 합침 (충돌이 나면 팀원과 같이 정리)
```

merge 후 **4장의 복원**을 하면 코드와 DB가 짝이 맞습니다.

Alembic은 따로 안 해도 되나요?

네. 덤프에 `alembic_version = 002_user_addresses` 가 이미 들어 있어서, 복원만 하면 팀원의 마이그레이션(사용자 주소 컬럼)까지 적용된 상태입니다. 나중에 팀원이 `003_…` 같은 새 마이그레이션을 추가하면 그때만 `alembic upgrade head` 를 실행하면 됩니다.

6. 자주 묻는 질문

Q. Releases 메뉴나 Assets 파일이 안 보여요.

팀 저장소는 **비공개(Private)**입니다. 팀 저장소 멤버로 초대받은 계정으로 로그인했는지 확인하세요. 로그인하지 않으면 저장소 자체가 404(없는 페이지)로 보입니다.

Q. 복원하면 내 DB에 있던 데이터는 어떻게 되나요?

③번에서 **기존 DB를 지우고 덤프 내용으로 바꿉니다.** 내가 테스트로 넣은 사용자(`user_profiles`) 같은 데이터도 같이 사라집니다. 남겨 두고 싶으면 복원 전에 내 DB를 먼저 덤프로 저장해 두세요.

```
docker compose exec -T db pg_dump -U cheongyak -d cheongyak -Fc -f /tmp/my_backup.dump
docker compose cp db:/tmp/my_backup.dump .\my_backup.dump
```

Q. 브랜치를 바꿀 때마다 다시 복원해야 하나요?

아니요. DB는 브랜치와 상관없이 내 컴퓨터에 하나만 있습니다. 브랜치를 바꿔도 DB는 그대로입니다. 새 덤프가 올라왔을 때만 다시 복원하면 됩니다.

Q. Release에 덤프가 여러 개면 무엇을 받나요?

날짜가 가장 최근인 것을 받으세요. 같은 날 두 번 만들면 `_2`처럼 번호가 붙습니다. 지금은 `cheongyak_20261006_2.dump`가 최신입니다.

Q. 복원 중에 오류 메시지가 나와요.

메시지	원인과 해결
`database "cheongyak" is being accessed by other users`	backend가 DB를 쓰는 중. `docker compose stop backend` 후 다시
`no such file or directory` (②번)	덤프 파일이 cheongyak-platform 폴더에 없거나 파일 이름이 다름. `dir *.dump`로 확인
`service "db" is not running`	Docker Desktop이 꺼져 있음. 켠 뒤 `docker compose up -d db`
`relation … already exists`	③번(DB 지우고 새로 만들기)을 건너뜀. ③번부터 다시

Q. 덤프는 누가, 언제 새로 올리나요?

DB 구조가 바뀌거나 데이터를 크게 고쳤을 때 지윤(DB 담당)이 새 Release를 추가하고 팀에 알립니다. 평소에는 각자 컴퓨터의 오전 6시 자동 수집으로 데이터가 최신으로 유지됩니다.

2026-10-06 · 팀 저장소 JiYoon241111/cheongyak-platform · 개인 백업 JiYoon241111/cheongyak-archive (db-version 브랜치, 같은 덤프)